フィードバック学習ループ設計

位置づけ: 本書はd_04 Agent OSの§7 Evaluation Engine / §18 品質改善ループに統合。詳細版として残す。

目的: 「使えば使うほど精度が上がる」を全モジュールで実現する
設計の正: このドキュメント（各設計書の学習ループ関連を統合）
参照元: c_02, b_01, e_01, d_01, c_03

実装状況（2026-03-18更新）: Sprint 1-4 全完了。実データ検証済み。
- DB migration: 012_feedback_learning.sql + 013_llm_call_logs.sql 適用済み
- API: finalize, rate, accuracy, stats, extraction-feedback 全5エンドポイント実装
- フロント: 確定ボタン + 👍👎 ボタン実装
- 実データ: 15PDF・718件の単価データをunit_price_masterに投入済み
- テスト: 134 passed (コアテスト全パス)

---

0. 設計原則

```
1. Day 1から計測する。計測できないものは改善できない
2. フィードバック収集 → 蓄積 → 活用 の3段階で段階的に実装
3. 個社レベルの学習が先。クロス企業学習（ネットワーク効果）は後
4. 人間の確認（Human-in-the-Loop）は常に残す。完全自動化はPhase 3+
```

---

1. 学習ループの全体像

```
Level 1: フィードバック収集（MVP。今回実装する）
  → ユーザーの修正・評価を記録する仕組み

Level 2: 蓄積データの活用（Month 2-3）
  → 蓄積データから精度を改善する仕組み

Level 3: クロス企業学習（Phase 2+）
  → N社の匿名化データでネットワーク効果を発揮
```

---

2. モジュール別 学習ループ設計

2.1 建設BPO — 積算AI

Level 1: フィードバック収集（★今回実装）

```
■ 単価フィードバック

  フロー:
    AI が単価を推定（confidence付き）
      → ユーザーが確認・修正
      → 「確定」ボタンを押す
      → 確定データが unit_price_master に保存される

  必要な実装:
    □ POST /estimation/projects/{id}/finalize エンドポイント
      - estimation_items の確定済みフラグを更新
      - learn_from_result() を呼び出し
      - 推定値 vs 確定値の乖離率を記録

    □ unit_price_master への保存項目:
      - category, subcategory, detail, specification, unit（工種情報）
      - unit_price（確定単価）
      - region, year（地域・年度）
      - source = "past_estimation"
      - source_detail = "プロジェクト名: {name}"
      - ai_estimated_price（AI推定値。乖離率計算用）
      - accuracy_rate = 1 - abs(ai_estimated - actual) / actual

    □ estimation_items への追記カラム:
      - user_modified BOOLEAN DEFAULT false
      - original_ai_price DECIMAL（AIの初期推定値を保存）
      - finalized_at TIMESTAMPTZ

■ 数量抽出フィードバック

  フロー:
    AI が数量を抽出
      → ユーザーが確認・修正（行の追加/削除/修正）
      → 修正内容が記録される

  必要な実装:
    □ extraction_feedback テーブル（新規）:
      CREATE TABLE extraction_feedback (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        company_id UUID NOT NULL REFERENCES companies(id),
        project_id UUID NOT NULL,
        original_items JSONB NOT NULL,    -- AI抽出結果
        corrected_items JSONB NOT NULL,   -- ユーザー修正後
```

```
    diff_summary JSONB,               -- 差分（追加/削除/修正の件数）
    source_format TEXT,               -- excel / pdf / text
    created_at TIMESTAMPTZ DEFAULT now()
  );

□ POST /estimation/projects/{id}/feedback エンドポイント
  - 抽出結果とユーザー修正後の差分を保存
  - diff_summary: {added: N, deleted: N, modified: N, total: N, accuracy: 0.XX}

■ 工種分類の正規化辞書

フロー:
  AI が「BH掘削」を抽出
    → ユーザーが「バックホウ掘削」に修正
    → 正規化辞書に追加される

必要な実装:
  □ term_normalization テーブル（新規）:
    CREATE TABLE term_normalization (
      id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
      company_id UUID REFERENCES companies(id),  -- NULLなら全社共通
      domain TEXT NOT NULL DEFAULT 'construction', -- 業界
      original_term TEXT NOT NULL,                 -- 元の表記
      normalized_term TEXT NOT NULL,               -- 正規化後
      occurrence_count INTEGER DEFAULT 1,          -- 出現回数
      created_at TIMESTAMPTZ DEFAULT now(),
      UNIQUE(company_id, domain, original_term)
    );

  □ 数量抽出時に正規化辞書を参照する処理を追加
  □ ユーザー修正時に辞書を自動更新
```

Level 2: 蓄積データの活用（Month 2-3）

```
■ 単価推定の精度向上

  現状: 過去実績を直近3件取得するだけ
  改善:
    □ 加重平均（直近重視）:
      weight = 1 / (1 + months_ago * 0.1)
      weighted_price = Σ(price × weight) / Σ(weight)

    □ confidence の動的計算:
      base_confidence = 0.5
      + 実績件数ボーナス: min(count * 0.05, 0.3)
      + 地域一致ボーナス: 0.1
      + 年度一致ボーナス: 0.05
      − 乖離率ペナルティ: avg_accuracy_rate < 0.8 → −0.1
      → 最終confidence = clip(0.1, 0.95)

    □ 推定精度のダッシュボード:
      − 工種別の平均accuracy_rate
      − confidence分布
      − 「精度が低い工種」アラート

■ 数量抽出の精度向上

  □ extraction_feedback から修正パターンを分析
    − 「Excel列Aが工種、列Bが種別」のようなフォーマット学習
    − よくある修正パターン → LLMプロンプトに追加
  □ 正規化辞書の自動拡充
    − occurrence_count が3以上 → 全社共通辞書に昇格
```

Level 3: クロス企業学習（Phase 2+）

```
  □ 匿名化された単価データの集約
    − k=5（5社以上のデータがある工種のみ公開）
    − 「同地域・同工種の市場単価: ¥XX,XXX（N=Y社）」
  □ 落札率予測（公共工事の予定価格との比較）
  □ テンプレート修正の集約（3社が同じ修正→テンプレート反映）
```

2.2 ブレイン — Q&A

Level 1: フィードバック収集（★今回実装）

```
■ ユーザー評価

  フロー:
    Q&A回答が表示される
      → ユーザーが 👍 or 👎 を押す
      → qa_sessions に評価が記録される

  必要な実装:
    □ qa_sessions テーブルへのカラム追加:
      ALTER TABLE qa_sessions ADD COLUMN user_rating INTEGER;
        -- NULL=未評価, 1=良い, -1=悪い
      ALTER TABLE qa_sessions ADD COLUMN rating_comment TEXT;
        -- 任意のコメント（「古い情報だった」等）
      ALTER TABLE qa_sessions ADD COLUMN source_item_ids UUID[];
        -- 回答に使われたknowledge_itemsのID群

    □ PATCH /knowledge/qa/{session_id}/rate エンドポイント
      - user_rating と rating_comment を更新
      - source_item_ids があれば、各itemのusage_countを更新

    □ フロントエンドに 👍👎 ボタン追加（Q&A回答の下）

■ 回答品質の計測

  □ 週次バッチ: 評価済みセッションの集計
    - positive_rate = 👍数 / 評価済み数
    - 目標: Phase 1で positive_rate ≥ 60%
  □ 低評価が多いナレッジの特定
    - source_item_ids × user_rating のクロス集計
    - 「このナレッジが使われた回答は評価が低い」→ 改善候補
```

Level 2: 蓄積データの活用（Month 2-3）

```
□ ナレッジの信頼度を動的更新
  - knowledge_items.confidence を Q&A評価で調整
  - 👍が多いitem → confidence UP
  - 👎が多いitem → confidence DOWN → 「要確認」フラグ

□ 「不足ナレッジ」の検出
  - 👎の rating_comment を分析
  - 「この質問に答えるナレッジがない」パターンを検出
  - → ナレッジ追加の提案

□ ベクトル検索のリランキング
  - 過去に👍が多かったitemを優先的にヒットさせる
  - usage_count × positive_rate でブースト
```

2.3 ブレイン — ナレッジ抽出

Level 1: フィードバック収集（★今回実装）

```
■ 抽出結果の評価

  フロー:
    テキスト/音声からナレッジを抽出
      → ユーザーが確認・修正
      → 修正内容が記録される

  必要な実装:
    □ knowledge_items テーブルへのカラム追加:
      ALTER TABLE knowledge_items ADD COLUMN extraction_modified BOOLEAN DEFAULT false;
      ALTER TABLE knowledge_items ADD COLUMN original_content JSONB;
        -- LLM抽出時の元データ（修正前を保存）

    □ 修正時に original_content を保存（未修正なら NULL）
    □ extraction_modified の割合を計測（抽出精度の指標）
```

2.4 BPO共通 — 承認ワークフロー学習

Level 1: フィードバック収集（★今回実装）

```
■ 承認・修正・却下の記録

  （d_01_BPOエージェントエンジン設計.md の Feedback Loop を実装）

  フロー:
    AI が作業結果を出す
      → 社長/管理者が 承認 / 修正して承認 / 却下
      → 結果と理由が記録される

  必要な実装:
    □ bpo_approvals テーブルへのカラム追加（既存テーブルに追記）:
      ALTER TABLE bpo_approvals ADD COLUMN modification_diff JSONB;
        -- 修正された場合の差分
      ALTER TABLE bpo_approvals ADD COLUMN rejection_reason TEXT;
        -- 却下理由
      ALTER TABLE bpo_approvals ADD COLUMN learned_rule TEXT;
        -- この承認/却下から学習したルール（LLMが生成）

    □ 承認時: trust_score +1
    □ 修正時: diffを記録、trust_score ±0（学習機会）
    □ 却下時: 理由を記録、trust_score −2
```

Level 2: 蓄積データの活用（Month 2-3）

```
  □ 修正diffからルール自動抽出（LLM）
    「社長は請求書の備考欄に工事番号を入れている」
    → knowledge_items に新ルールとして追加提案
    → 社長が確認 → 承認されたら反映

  □ Trust Scorer による自律度レベル管理
    Level 0: 通知のみ
    Level 1: 下書き作成（承認必須）
    Level 2: 自動実行（事後レビュー）
    Level 3: 完全自律（異常時のみ通知）

    昇格条件:
      0→1: 5回以上の実行 + SaaS接続済み
      1→2: 承認率80%以上 + 20回以上
      2→3: 承認率90%以上 + 30回連続成功 + 社長明示承認
```

2.5 ゲノム（テンプレート）— フィードバックループ

Level 1: 修正追跡（★今回実装）

```
□ テンプレートから生成されたknowledge_itemsの修正を追跡
  - source_type = "template" のitemが修正された場合
  - original_content（テンプレート値）vs 現在値 の差分を記録

□ テンプレート適用後の修正率を計測
  - 修正率 = 修正されたitem数 / テンプレートから生成されたitem総数
  - 修正率が高い部門 → テンプレートの品質が低い → 改善候補
```

Level 3: クロス企業テンプレート進化（Phase 2+）

```
□ 3社が同じ修正 → テンプレートに反映
  - A社が「着工前チェックリストに近隣挨拶を追加」
  - B社も同じ修正
  - C社も同じ修正
  → テンプレート本体に「近隣挨拶」を追加
□ パターン提案:「同業他社の80%がこのルールを持っています」
```

---

3. DB変更一覧（今回のマイグレーション）

```sql
-- 012_feedback_learning.sql

-- 1. 積算 単価フィードバック
ALTER TABLE estimation_items ADD COLUMN user_modified BOOLEAN DEFAULT false;
ALTER TABLE estimation_items ADD COLUMN original_ai_price DECIMAL(15,2);
ALTER TABLE estimation_items ADD COLUMN finalized_at TIMESTAMPTZ;

ALTER TABLE unit_price_master ADD COLUMN ai_estimated_price DECIMAL(15,2);
ALTER TABLE unit_price_master ADD COLUMN accuracy_rate DECIMAL(5,4);
ALTER TABLE unit_price_master ADD COLUMN used_count INTEGER DEFAULT 0;

-- 2. 数量抽出フィードバック
CREATE TABLE extraction_feedback (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  project_id UUID NOT NULL,
  original_items JSONB NOT NULL,
  corrected_items JSONB NOT NULL,
  diff_summary JSONB,
  source_format TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);
ALTER TABLE extraction_feedback ENABLE ROW LEVEL SECURITY;

-- 3. 工種正規化辞書
CREATE TABLE term_normalization (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID REFERENCES companies(id),
  domain TEXT NOT NULL DEFAULT 'construction',
  original_term TEXT NOT NULL,
  normalized_term TEXT NOT NULL,
  occurrence_count INTEGER DEFAULT 1,
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(company_id, domain, original_term)
);

-- 4. Q&A フィードバック
ALTER TABLE qa_sessions ADD COLUMN user_rating INTEGER;
ALTER TABLE qa_sessions ADD COLUMN rating_comment TEXT;
ALTER TABLE qa_sessions ADD COLUMN source_item_ids UUID[];

-- 5. ナレッジ抽出フィードバック
ALTER TABLE knowledge_items ADD COLUMN extraction_modified BOOLEAN DEFAULT false;
ALTER TABLE knowledge_items ADD COLUMN original_content JSONB;

-- 6. BPO承認フィードバック
ALTER TABLE bpo_approvals ADD COLUMN modification_diff JSONB;
```

```
ALTER TABLE bpo_approvals ADD COLUMN rejection_reason TEXT;
ALTER TABLE bpo_approvals ADD COLUMN learned_rule TEXT;
```

---

4. API変更一覧（今回追加するエンドポイント）

メソッド	パス	機能
POST	`/estimation/projects/{id}/finalize`	積算確定 + 単価学習
POST	`/estimation/projects/{id}/feedback`	数量抽出フィードバック保存
GET	`/estimation/accuracy`	推定精度ダッシュボード
PATCH	`/knowledge/qa/{session_id}/rate`	Q&A評価（👍👎）
GET	`/knowledge/qa/stats`	Q&A品質統計

---

5. フロントエンド変更一覧

画面	変更内容
積算ウィザード Step 4（単価設定）	AI推定値の表示 + 修正UI + 「確定」ボタン
積算一覧	精度統計の表示（accuracy_rate）
Q&A画面	回答の下に 👍👎 ボタン + コメント欄
ダッシュボード	学習進捗（単価蓄積数、Q&A評価率、抽出精度）

---

6. 実装優先順位

```
Sprint 1 ✅ 完了（2026-03-18）: 最小限の学習ループ
  ✅ DB migration 012_feedback_learning.sql（Supabase適用済み）
  ✅ DB migration 013_llm_call_logs.sql（Supabase適用済み）
  ✅ POST /estimation/projects/{id}/finalize（単価学習）
  ✅ PATCH /knowledge/qa/{session_id}/rate（Q&A 👍👎）
  ✅ フロントに「確定」ボタンと「👍👎」ボタン
  テスト: 134 passed, 0 failed

Sprint 2 ✅ 完了（2026-03-18）: 計測基盤
  ✅ GET /estimation/accuracy（精度ダッシュボード）
  ✅ GET /knowledge/qa/stats（Q&A品質統計）
  ✅ POST /estimation/projects/{id}/extraction-feedback（フィードバック保存）
  テスト: 134 passed, 0 failed

Sprint 3 ✅ 完了（2026-03-18）: 正規化辞書 + Level 2の基礎
  ✅ term_normalization テーブル + extraction-feedback時の自動登録
  ✅ suggest_unit_prices の加重平均化（直近重視）
  ✅ confidence の動的計算（実績件数・地域一致・精度ペナルティ）
  ✅ used_count 更新（単価参照時にカウント+1）
  テスト: 134 passed, 0 failed

Sprint 4 ✅ 完了（2026-03-18）: BPO承認ワークフロー学習
  ✅ bpo_approvals の拡張（modification_diff, rejection_reason, learned_rule）
  ✅ TrustScorer（承認率・連続成功でレベル自動昇格 0→1→2→3）
  ✅ approve_with_learning（修正diff→LLMルール抽出）
  テスト: 134 passed, 0 failed

実データ検証 ✅ 完了（2026-03-18）:
  ✅ 中部地方整備局 設計書PDF 5件で初回検証
  ✅ 関東地方整備局（東京国道・川崎国道）設計書PDF 10件を追加
  ✅ 合計15PDF処理、12件成功（80%）、718件の単価データ抽出
  ✅ unit_price_masterに718件を初期投入（scripts/seed_unit_prices.py）
  ✅ 積算AI処理: 1.6秒/PDF、API呼び出し0回、コスト¥0
  ✅ 工種分類精度: 96.3%（18カテゴリ・キーワードルール）
  ✅ 失敗3件の原因: スキャン画像PDF（OCR未対応。Phase 2+）
```

7. 成功指標

指標	Phase 1目標	Phase 2目標
単価推定 accuracy_rate	≥ 0.60	≥ 0.80
数量抽出 正解率	≥ 0.70	≥ 0.85
Q&A positive_rate	≥ 0.60	≥ 0.80
テンプレート 修正率	≤ 0.40	≤ 0.20
BPO承認率	≥ 0.70	≥ 0.90
単価マスタ蓄積数/企業	≥ 50件	≥ 500件